

Relatório de Laboratório

Lab01 — Características de Repositórios Populares + Setup do Kanban

Campo	Informação
Curso	Engenharia de Software
Disciplina	Laboratório de Experimentação de Software
Turno / Período	Noite / 6º
Professor(a)	Danilo Maia
Laboratório	Lab01 — Mineração repositórios populares
Grupo (trio)	Rafael F. Franco · Paulo Assis · Vitor Rebula
Repositório / GitHub Projects	https://github.com/RafaelFFranco/Laboratorio-Medicao-e-Experimentacao-de-Software · Board (Projects): https://github.com/users/RafaelFFranco/projects/3
Data de entrega	26/08/2026

Laboratório de Experimentação de Software

1. Introdução

Repositórios open-source de grande visibilidade concentram boa parte da atividade de desenvolvimento colaborativo da indústria, mas a intuição sobre suas características se são maduros, se recebem muita contribuição, se são bem mantidos raramente é confrontada com dados. Este laboratório investiga empiricamente as principais características dos 1.000 repositórios com maior número de estrelas do GitHub, aplicando mineração de dados via API sobre uma amostra ampla, em vez de partir de percepções isoladas. O tema importa porque essas características (idade, contribuição externa, cadência de releases, manutenção, linguagem e disciplina de issues) são exatamente os indicadores que times e empresas usam para julgar a saúde e a confiabilidade de uma dependência antes de adotá-la.

Além da parte de mineração, o laboratório inicia o uso do quadro Kanban (GitHub Projects v2) que acompanhará o grupo em todos os laboratórios seguintes, servindo de base de dados de processo para os Labs 04 e 05. As Questões de Pesquisa do enunciado (RQ01–RQ06, mais o bônus RQ07), são apresentadas a seguir, cada uma acompanhada da hipótese informal do grupo levantada antes de olhar os dados.

Questões de Pesquisa e hipóteses informais

RQ	Pergunta do enunciado	Hipótese informal do grupo
RQ01	Sistemas populares são maduros/antigos?	Sim — esperamos repositórios com vários anos de idade; popularidade se acumula com o tempo.
RQ02	Sistemas populares recebem muita contribuição externa?	Sim — projetos populares atraem muitos contribuidores e devem ter grande volume de PRs aceitas.
RQ03	Sistemas populares lançam releases com frequência?	Sim — projetos maduros tendem a versionar de forma organizada e frequente.
RQ04	Sistemas populares são atualizados com frequência?	Sim — esperamos atualização recente (poucos dias/semanas desde o último push).
RQ05	Sistemas populares usam as linguagens mais populares?	Sim, majoritariamente — a maioria deve estar entre as linguagens líderes do ranking de referência.
RQ06	Sistemas populares têm alto percentual de issues fechadas?	Sim — projetos bem mantidos devem fechar a maior parte de suas issues.
RQ07 (bônus)	Linguagens mais populares recebem mais contribuição, mais releases e são atualizadas com mais frequência?	Parcialmente — esperamos vantagem das linguagens de ecossistema web/JS, mas sem diferença dramática.

Contribuições próprias do grupo

- **RQ-Extra 1** — nova RQ correlacional: existe relação entre a diversidade de linguagens de um repositório (nº de linguagens distintas) e o volume de contribuição externa (PRs aceitas)?
- **Métrica adicional** — cálculo do coeficiente de correlação de Pearson (r , p -valor) para a RQ-Extra, indo além das medianas descritivas pedidas no enunciado.
- **Arquitetura** — coleta resiliente com checkpoint em disco, retomada automática após rate limit e backoff exponencial escrito do zero (sem biblioteca de terceiros para a API), permitindo coletar os 1.000 repositórios sem reiniciar do início a cada falha.

2. Contexto

O Lab01 correspondeu ao primeiro laboratório do semestre e marcou o ponto de partida do processo de mineração de dados e da organização do trabalho da equipe. A partir dele, o projeto evoluiu de forma incremental ao longo das sprints, com cada etapa contribuindo para a construção dos resultados finais.

Na Sprint 1, iniciamos o projeto com a mineração dos dados do GitHub, desenvolvendo e estruturando o script de consulta utilizando a API GraphQL. Na Sprint 2, o trabalho avançou com a implementação da paginação das consultas e a organização dos dados minerados em um arquivo CSV, permitindo maior volume e consistência na análise. Já nas Sprints 3 e 4, concentramos os esforços no tratamento e na análise dos dados, chegando à consolidação dos resultados e à elaboração dos relatórios, que se encontram atualmente finalizados.

Paralelamente ao desenvolvimento técnico, mantivemos um quadro Kanban durante todo o projeto, utilizando-o para acompanhar as atividades, distribuir responsabilidades e organizar o fluxo de trabalho da equipe. Ao longo das sprints, os snapshots do quadro foram atualizados, permitindo registrar a evolução do processo e servindo como uma base para acompanhar como as atividades foram desenvolvidas e organizadas durante o semestre.

Dessa forma, o Lab01 não apenas iniciou a mineração dos dados, mas também estabeleceu a base para a evolução contínua do projeto, que atualmente se encontra na Sprint 4, com os dados minerados, processados e os relatórios concluídos, além de todo o histórico de organização e acompanhamento do trabalho preservado por meio do Kanban e de seus snapshots.

O objeto de estudo são os 1.000 repositórios com maior número de estrelas do GitHub, obtidos pela busca ordenada por estrelas (query "stars:>1 sort:stars-desc"). Para cada repositório coletamos idade, pull requests aceitas, total de releases, dias desde a última atualização, linguagem primária e razão de issues fechadas. Como referência conceitual para "linguagens mais populares" (RQ05), o grupo adotou o GitHub Octoverse 2025 que mede o número de desenvolvedores distintos por mês que enviaram código em cada linguagem, no período de 01/09/2024 a 31/08/2025 e manteve essa mesma fonte de forma consistente do início ao fim do laboratório. A escolha do Octoverse (em vez de TIOBE ou GitHub) se justifica por ele medir atividade real de desenvolvedores dentro do próprio GitHub, que é o universo da amostra.

3. Metodologia

3.1 Principais Desafios

- **Rate limit da API GraphQL** - a busca por repositórios paginada consome cota rapidamente. A query pede rateLimit (remaining, resetAt, cost) em toda requisição e o cliente pausa proativamente quando restam menos de 5 pontos, aguardando até o resetAt.
- **Paginação de 1.000 repositórios** - o endpoint de busca do GitHub não retorna 1.000 itens de uma vez. Foi necessário paginar com cursores (pageInfo.endCursor / hasNextPage) em páginas de 50, deduplicando por nameWithOwner para não contar o mesmo repositório duas vezes.
- **Falhas parciais durante a coleta** - coletar 1.000 repositórios pode falhar no meio (erro de servidor, corte de conexão, rate limit). Reiniciar do zero a cada falha é inviável, o que motivou o mecanismo de checkpoint descrito em 3.6.
- **Definição operacional da RQ06** - repositórios como torvalds/linux, git/git e django/django não usam o issue tracker do GitHub (0 issues). A razão issues fechadas/total viraria 0/0 (indefinida). Decidiu-se tratá-los como valor ausente, e não como 0%, para não subestimar artificialmente a métrica.

- **Repositórios sem linguagem primária** - 87 dos 1.000 repositórios não têm linguagem primária definida na API (repositórios de documentação, listas, dotfiles). Precisaram ser separados dos que têm linguagem para não distorcer os percentuais da RQ05.

3.2 Tomadas de Decisão

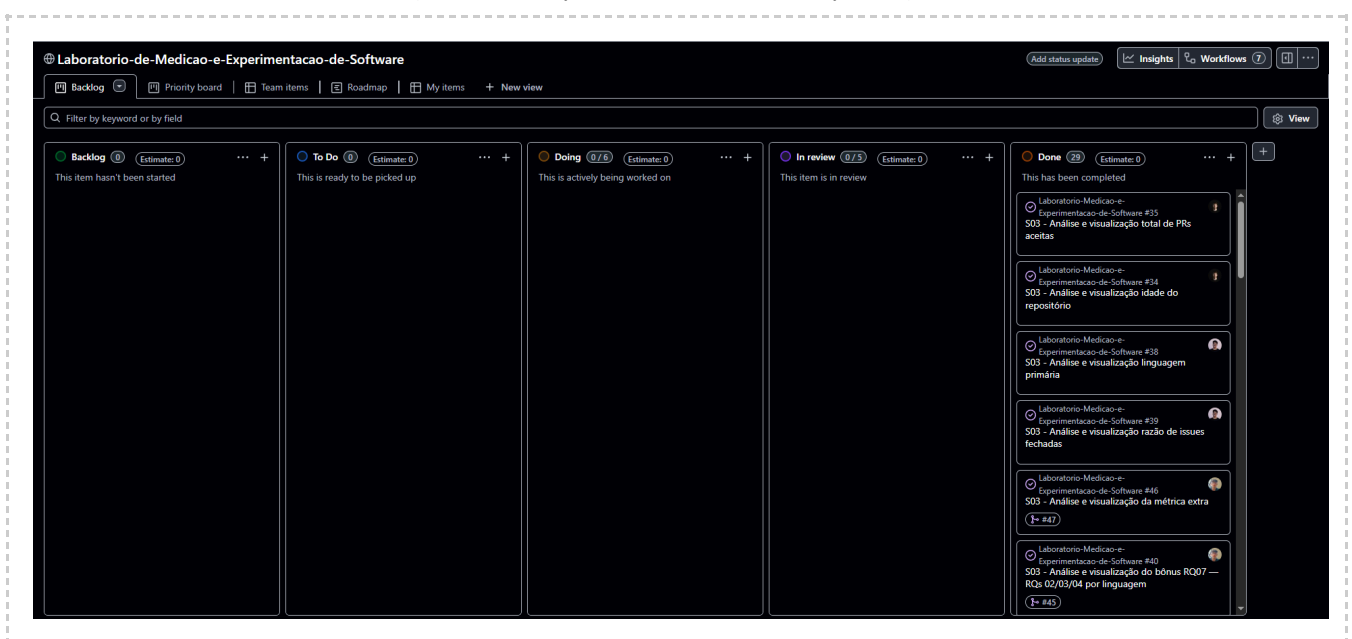
- **Limite de WIP na coluna Doing** - definido em 2 cartões simultâneos por integrante. Justificativa: trio com desenvolvimento individual semanal; um limite baixo força a finalização (mover para Review/Done) antes de puxar tarefa nova, evitando trabalho pela metade acumulado no fim da sprint.
- **Definição de razão de issues fechadas** - adotada como issues fechadas / total de issues, calculada apenas sobre repositórios com pelo menos 1 issue; repositórios com 0 issues são excluídos do numerador e do denominador (valor ausente), decisão mantida idêntica para toda a amostra.
- **Fonte de "linguagens mais populares"** - GitHub Octoverse 2025 escolhido e fixado desde o início, com citação completa (métrica, período, data de publicação e acesso) impressa no rodapé de todos os gráficos da RQ05, garantindo reprodutibilidade.
- **Mediana em vez de média** — para as métricas numéricas (idade, PRs, releases, dias desde atualização, razão de issues) usou-se a mediana como medida de tendência central, por serem distribuições fortemente assimétricas e com outliers extremos (ex.: PRs aceitos vão de dezenas a mais de 100 mil). Para a RQ05, por ser categórica, usou-se a moda/contagem.
- **Linguagem do pipeline** — Python foi escolhido por ser adequado tanto para a coleta (requests + GraphQL manual) quanto para a análise (Pandas) e visualização (Matplotlib/Seaborn), mantendo todo o pipeline em uma única linguagem.

3.3 Etapas

O desenvolvimento seguiu a estrutura de sprints do enunciado.

Configuração do processo (GitHub Projects v2)

- **Colunas (Status):** Backlog → To Do → Doing → Review → Done.
- **Política de WIP:** 2 cartões em Doing por integrante (ver justificativa em 3.2).
- **Rastreabilidade:** cada tarefa é uma Issue de verdade (não draft), com Assignee, e cada commit referencia o nº da Issue (ex.: "#12 implementa consulta GraphQL").



3.4 Ferramentas

- **GitHub GraphQL API** - endpoint <https://api.github.com/graphql>, consumido por script próprio do grupo com a biblioteca requests (sem biblioteca de terceiros que encapsule a API do GitHub, conforme o enunciado).
- **Python 3 + Pandas / NumPy** - para leitura e manipulação do CSV de 1.000 repositórios.
- **Matplotlib + Seaborn** - para os gráficos (boxplots, histogramas, barras de ranking, heatmap, dispersão).
- **SciPy** - `scipy.stats.pearsonr` para o coeficiente de correlação da RQ-Extra.
- **python-dotenv** - para carregar o token de acesso a partir de variável de ambiente.
- **GitHub Projects** - o board do grupo, com colunas de Status, política de WIP e Issues rastreáveis via API.

3.5 Tabela de Métricas

Cada RQ, sua métrica, definição operacional exata e a fonte de coleta:

RQ	Métrica	Definição Operacional	Unidade	Fonte
RQ01	Idade do repositório	(data atual – createdAt), convertida para anos.	anos	GraphQL
RQ02	PRs aceitas	<code>pullRequests(states: MERGED).totalCount</code> .	PRs	GraphQL
RQ03	Total de releases	<code>releases.totalCount</code> .	releases	GraphQL
RQ04	Dias desde última atualização	(data atual – pushedAt), em dias.	dias	GraphQL
RQ05	Linguagem primária	<code>primaryLanguage.name</code> ; comparada ao top-10 do Octoverse 2025.	categórica	GraphQL + Octoverse
RQ06	Razão de issues fechadas	<code>issues(CLOSED).totalCount ÷ issues.totalCount</code> ; 0 issues = ausente.	0–1	GraphQL
RQ07	Métricas RQ02–04 por linguagem	Mediana de PRs, releases e dias-de-atualização agrupada por linguagem primária.	por linguagem	GraphQL + Pandas
RQ-Extra	Diversidade × contribuição	Correlação de Pearson entre nº de linguagens distintas e PRs aceitas.	r (–1 a 1)	GraphQL + SciPy

3.6 Inovações Propostas pelo Grupo

(a) Nova Questão de Pesquisa (RQ-Extra). Além das RQs do enunciado, o grupo investigou se a diversidade de linguagens de um repositório (número de linguagens distintas presentes) se relaciona com o volume de contribuição externa (PRs aceitas). A motivação é que projetos políglotas tendem a ser maiores e a envolver mais subsistemas, o que poderia atrair mais contribuidores. O resultado aparece na seção 4.2 (gráfico de dispersão) e é discutido em 4.3.

(b) Métrica estatística adicional. Em vez de parar na descrição por medianas, calculamos o coeficiente de correlação de Pearson (r e p -valor) para a RQ-Extra, quantificando força e significância da relação.

(c) Arquitetura de coleta resiliente. O cliente GraphQL, escrito do zero, implementa: backoff exponencial para erros 5xx e rate limit secundário; pausa proativa quando a cota (`rateLimit.remaining`) cai abaixo de 5, aguardando até `resetAt`; redução automática do tamanho da página quando uma página falha repetidamente; e checkpoint em disco (`checkpoint_repos.json`) que salva o progresso a cada página e permite retomar a coleta exatamente de onde parou. Sem isso, coletar 1.000 repositórios sob rate limit seria impraticável.

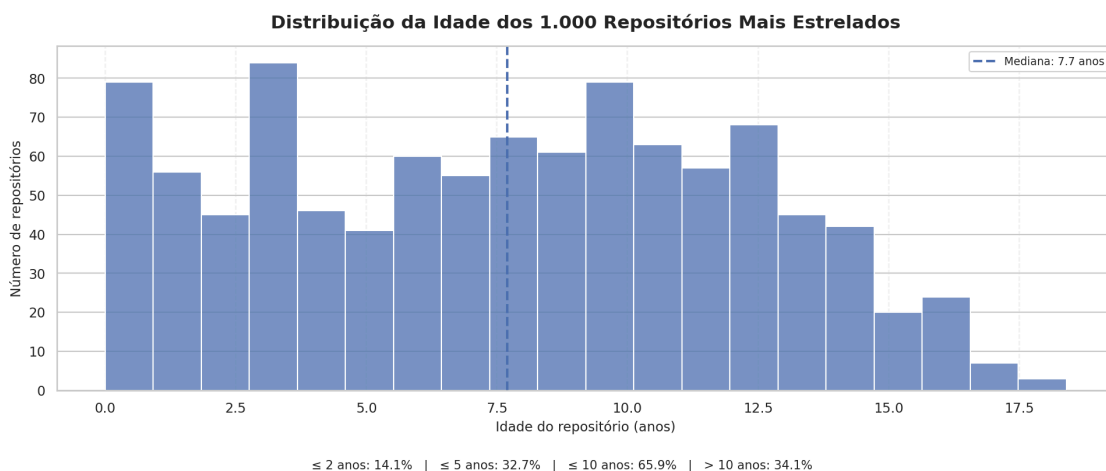
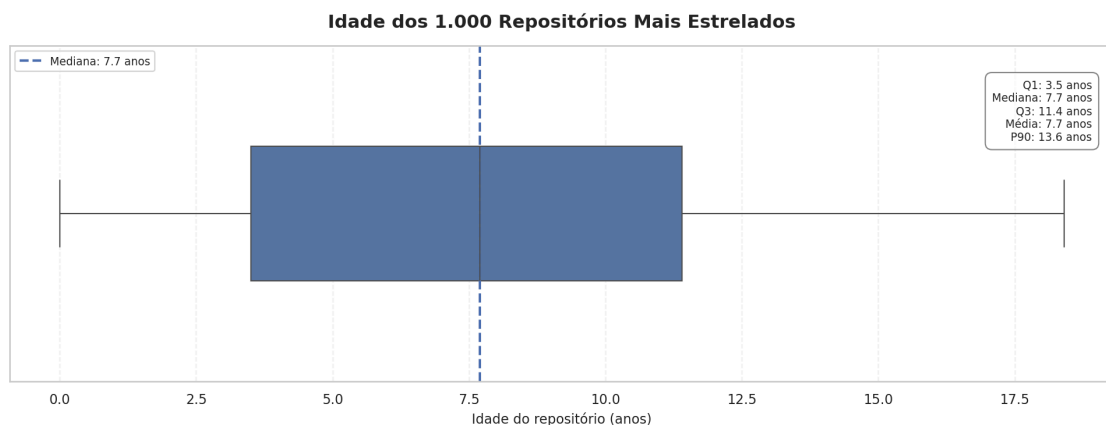
4. Resultados

4.1 Coleta de Dados

A coleta atingiu o alvo integral de 1.000 repositórios, exportados para o arquivo repositorios_1000.csv com 20 campos por repositório. A cobertura por métrica variou conforme a disponibilidade dos dados na API: as métricas de idade, PRs, releases e atualização estão presentes nos 1.000 repositórios; a linguagem primária está definida em 913 repositórios (87 sem linguagem, tipicamente repositórios de documentação e listas); e a razão de issues fechadas foi calculada sobre os 957 repositórios que possuem ao menos uma issue os 43 sem qualquer issue (ex.: torvalds/linux, git/git, django/django) foram tratados como valor ausente, não como 0%. Os dados refletem o estado do GitHub no momento da coleta (acesso em 25/08/2026).

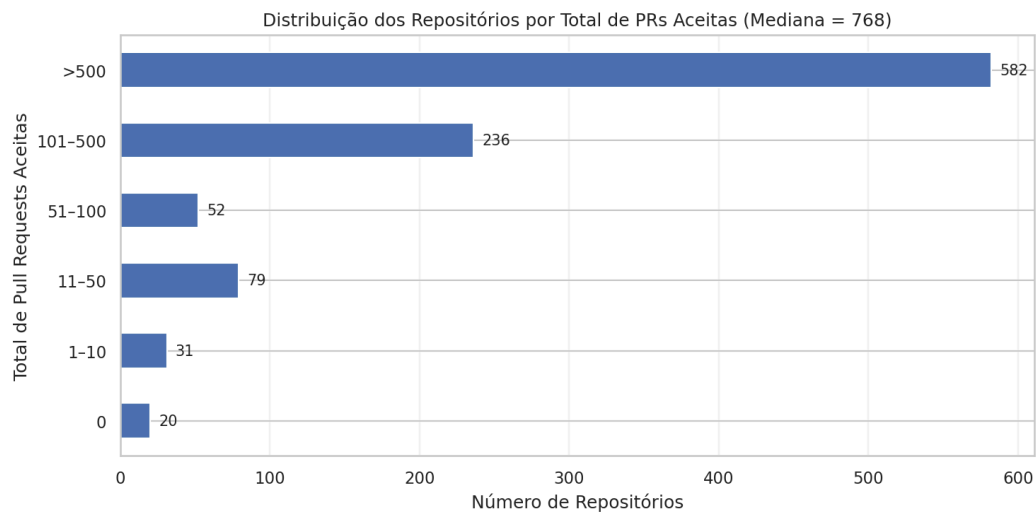
4.2 Visualização Gráfica

RQ01 — Sistemas populares são maduros/antigos?



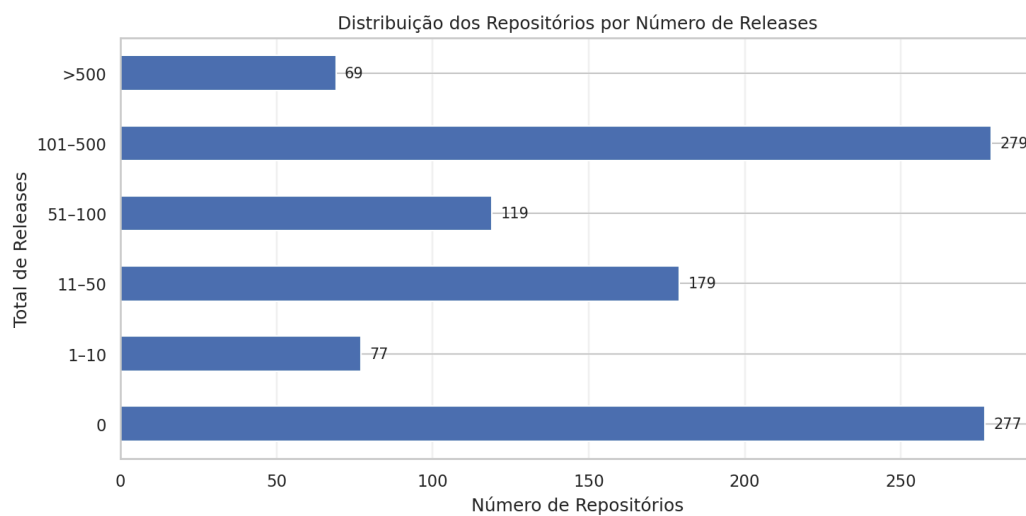
Idade dos 1.000 repositórios. Mediana = 7,7 anos; 34,1% têm mais de 10 anos e apenas 14,1% têm até 2 anos.

RQ02 — Sistemas populares recebem muita contribuição externa?



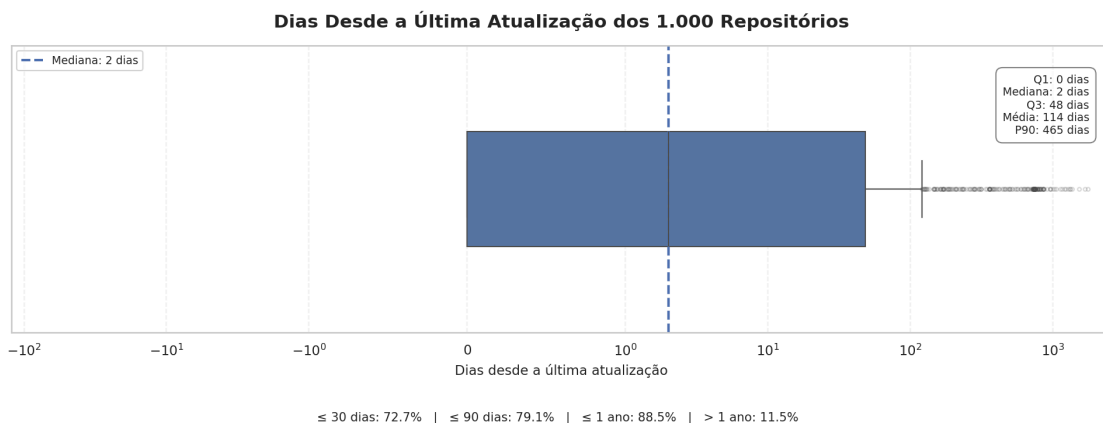
Distribuição de PRs aceitas. Mediana = 768 PRs aceitas (média puxada por outliers, até 103.370).

RQ03 — Sistemas populares lançam releases com frequência?



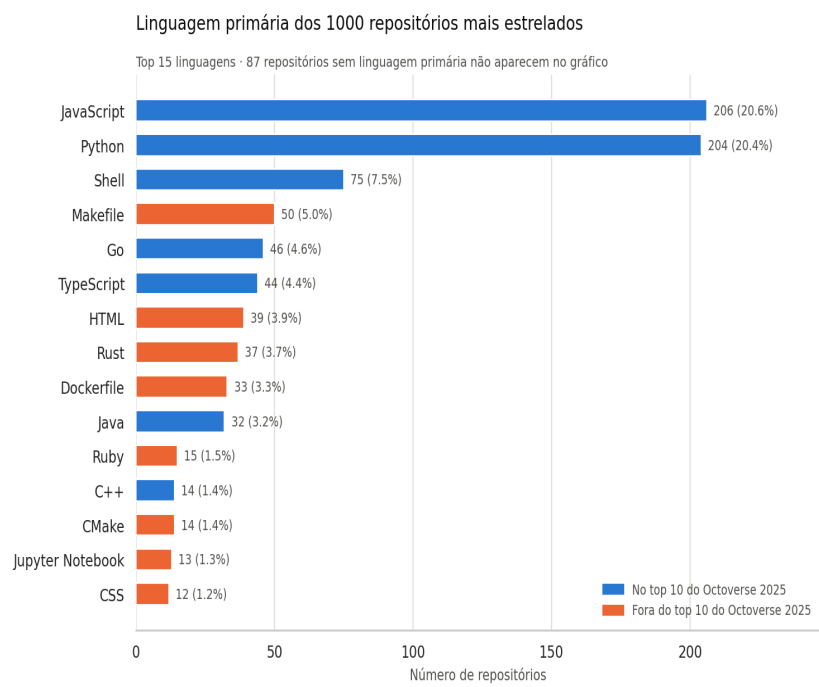
Distribuição do total de releases. Mediana = 40 releases; 27,7% dos repositórios têm 0 releases.

RQ04 — Sistemas populares são atualizados com frequência?

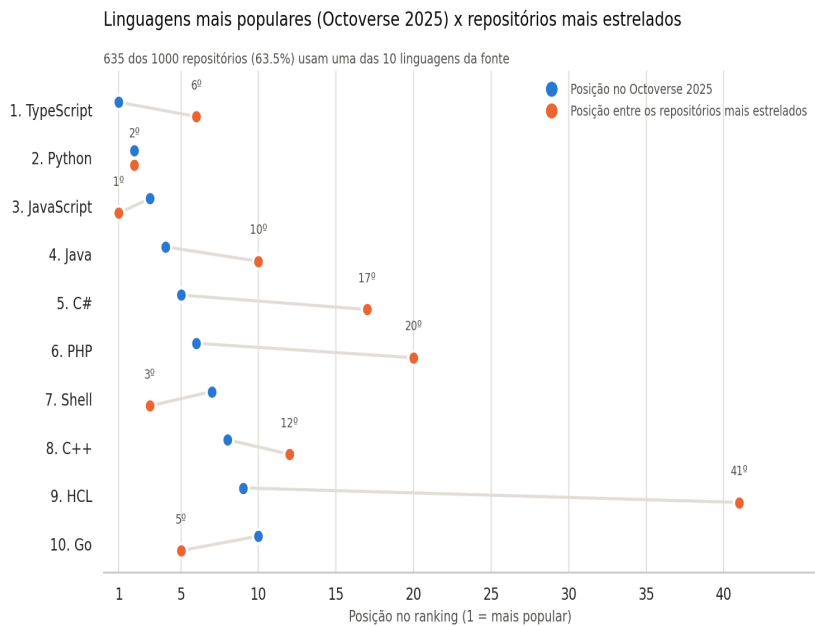


Dias desde a última atualização. Mediana = 2 dias; 72,7% foram atualizados nos últimos 30 dias.

RQ05 — Sistemas populares usam as linguagens mais populares?



Fonte de "linguagens mais populares": GitHub Octoberse 2025 (métrica: número de desenvolvedores distintos por mês que enviaram código na linguagem; período 01/09/2024 a 31/08/2025; publicado em 28/10/2025, acesso em 25/08/2026). <https://github.blog/news-insights/octoverse/octoverse-a-new-developer-joins-github-every-second-as-ai-leads-typescript-to-1/>



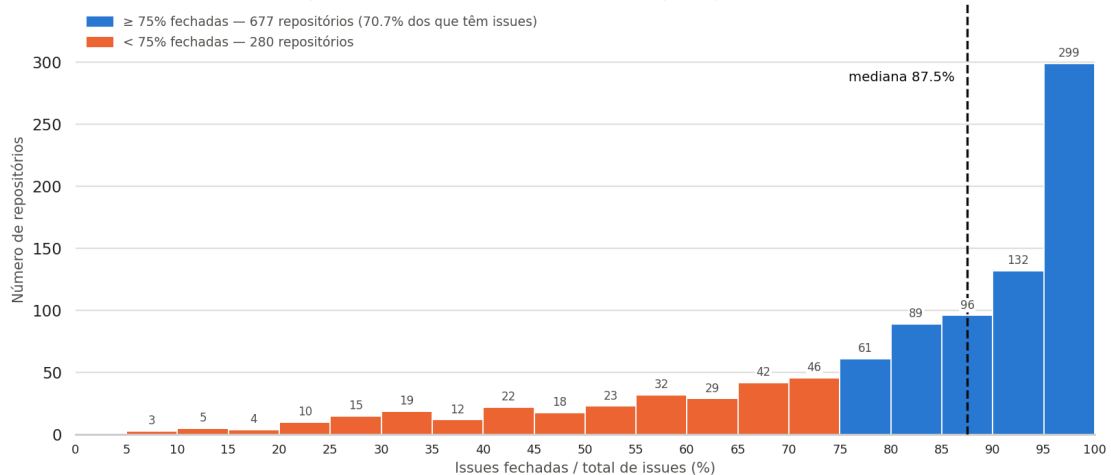
Fonte de "linguagens mais populares": GitHub Octoverse 2025 (métrica: número de desenvolvedores distintos por mês que enviaram código na linguagem; período 01/09/2024 a 31/08/2025; publicado em 28/10/2025, acesso em 25/08/2026).
<https://github.blog/news-insights/octoverse/octoverse-a-new-developer-joins-github-every-second-as-ai-leads-typescript-to-1/>

Linguagem primária vs. top-10 do Octoverse 2025. 635 dos 1.000 repositórios (63,5%) usam uma das 10 linguagens mais populares da fonte.

RQ06 — Sistemas populares têm alto percentual de issues fechadas?

Razão de issues fechadas dos 1000 repositórios mais estrelados

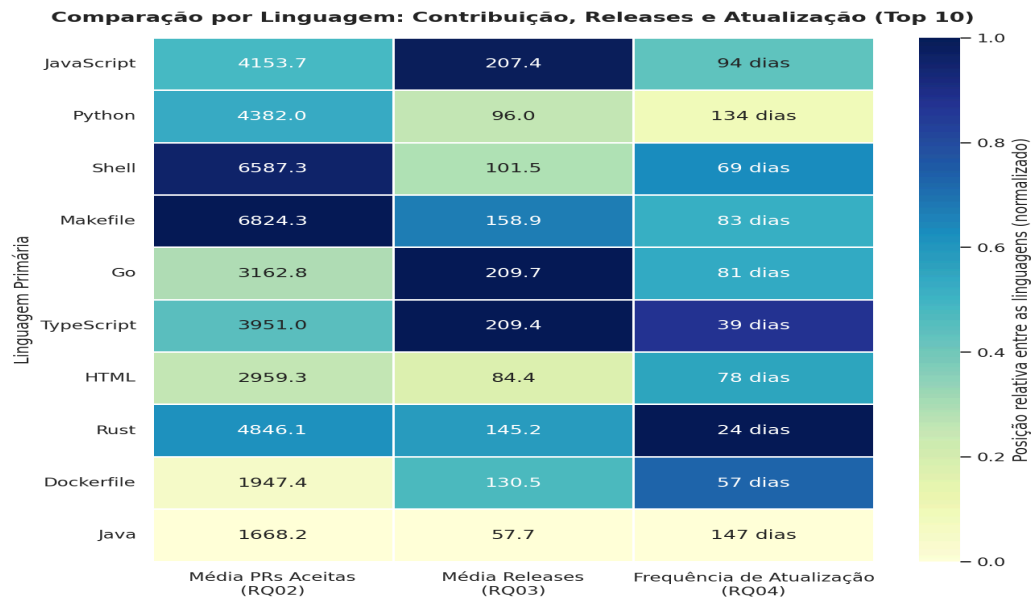
Mediana de 87.5% entre os 957 repositórios com issues - cada barra é uma faixa de 5 pontos percentuais



Razão = issues fechadas / total de issues (campos totalIssues e closedIssues da API GraphQL do GitHub).
43 dos 1000 repositórios têm 0 issues: a razão seria 0/0, indefinida, e por isso não entram na mediana nem no gráfico (ex.: torvalds/linux, git/git, django/django, que não usam o issue tracker do GitHub).

Razão de issues fechadas (957 repositórios com issues). Mediana = 87,5%; 70,7% fecham ao menos 75% das issues.

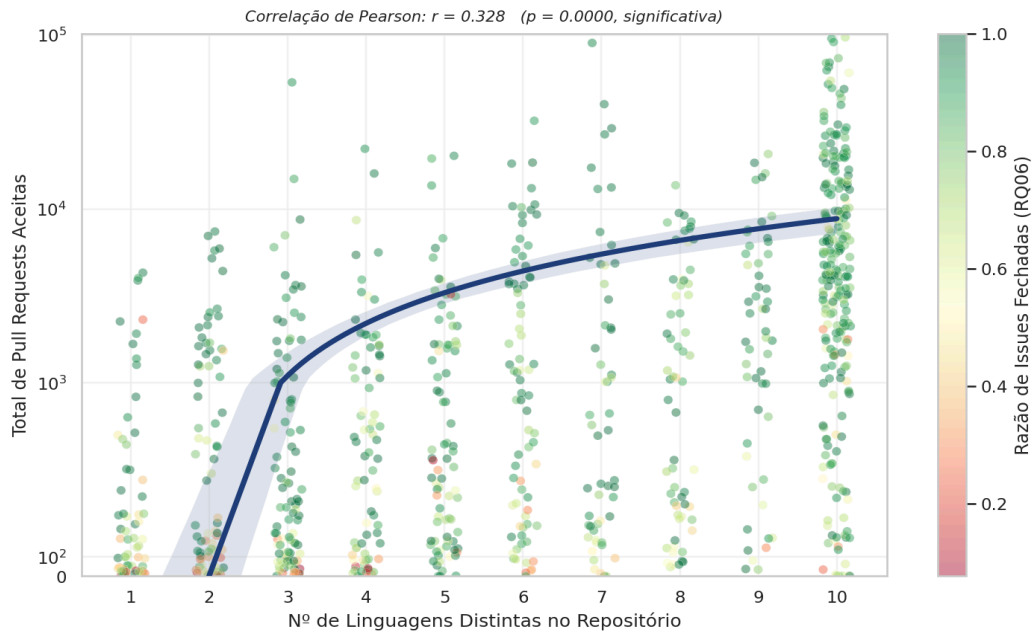
RQ07 (bônus) — Linguagens mais populares recebem mais contribuição, mais releases e são atualizadas com mais frequência?



Médias de PRs aceitas, releases e dias-de-atualização por linguagem (top-10 linguagens por contagem). Valores normalizados na cor; rótulos mostram o valor real.

RQ-Extra (inovação) — Há relação entre diversidade de linguagens e contribuição externa?

Diversidade Linguística vs. Contribuição Externa e Qualidade de Manutenção



4.3 Discussão

RQ	Hipótese	Resultado obtido e leitura
RQ01	Confirmada	Mediana de 7,7 anos e 34% acima de 10 anos confirmam que repositórios populares são maduros; a popularidade se acumula ao longo do tempo.
RQ02	Confirmada	Mediana de 768 PRs aceitas indica contribuição externa expressiva; a distribuição é muito assimétrica (poucos projetos concentram dezenas de milhares).
RQ03	Parcialmente confirmada	Mediana de 40 releases mostra versionamento frequente na maioria, mas 27,7% não têm nenhuma release — muitos projetos populares (docs, listas, frameworks grandes) não usam releases formais.
RQ04	Confirmada	Mediana de apenas 2 dias e 72,7% atualizados no último mês confirmam manutenção muito ativa.
RQ05	Parcialmente confirmada	63,5% usam uma das 10 linguagens mais populares do Octoverse — maioria, mas não esmagadora; há forte cauda de linguagens e 8,7% sem linguagem definida.
RQ06	Confirmada	Mediana de 87,5% de issues fechadas confirma alta disciplina de manutenção entre os que usam o issue tracker.
RQ07	Parcialmente confirmada	Nenhuma linguagem lidera nas três métricas ao mesmo tempo: em média, Go e TypeScript lançam mais releases; Rust é a mais atualizada (menor nº de dias); e linguagens de script (Makefile, Shell) concentram os maiores volumes de PRs aceitas. Não há vantagem única e clara das linguagens "mais populares".

Sobre a inovação (RQ-Extra): o coeficiente de Pearson $r = 0,328$ ($p < 0,001$) indica correlação positiva, de intensidade fraca a moderada, e estatisticamente significativa entre diversidade de linguagens e PRs aceitas ou seja, repositórios com mais linguagens tendem a receber mais contribuições, mas a relação está longe de ser determinística ($r^2 \approx 0,11$, a diversidade explica cerca de 11% da variação). A inovação aprofunda a RQ02: mostra que o alto volume de contribuição não se distribui de forma homogênea e está parcialmente associado à complexidade poliglota do projeto.

Ameaças à validade. (i) Estrelas medem a popularidade percebida, não uso real, repositórios de listas e tutoriais entram na amostra e distorcem métricas de manutenção. (ii) A API conta PRs e releases totais, sem distinguir contribuição de mantenedores da externa. (iii) A exclusão dos repositórios sem issues na RQ06 é conservadora, mas assume que ausência de issues = ausência de dado, não baixa manutenção. (iv) A coleta é um retrato de um instante (25/08/2026); executar em outra data produziria valores ligeiramente diferentes.

5. Conclusão

Os dados dos 1.000 repositórios mais estrelados desenharam um perfil consistente: são projetos maduros (mediana de ~7,7 anos), muito ativos (mediana de 2 dias desde a última atualização), com contribuição externa expressiva (mediana de 768 PRs aceitas) e alta disciplina de issues (mediana de 87,5% fechadas). As hipóteses de idade, contribuição, atualização e issues foram confirmadas; as de releases e de linguagem foram apenas parcialmente confirmadas, revelando que "repositório popular" é uma categoria mais

heterogênea do que a intuição sugere inclui bibliotecas versionadas com rigor, mas também repositórios de documentação e listas que não usam releases nem issue tracker.

A principal limitação é que a estrela é um proxy imperfeito de relevância, e a coleta é um retrato de um único instante. A inovação do grupo a RQ correlacional com teste de Pearson ($r = 0,328$, $p < 0,001$) vale ser expandida em um laboratório futuro: seria interessante controlar por tamanho do repositório e por idade para separar o efeito da diversidade de linguagens do simples efeito "projetos maiores recebem mais de tudo". Com mais tempo, o grupo também segmentaria as métricas por domínio de aplicação (web, ML, infraestrutura) e coletaria uma segunda amostra em data diferente para medir estabilidade temporal das métricas.

6. Referências

GITHUB. *Octoverse 2025: a new developer joins GitHub every second as AI leads TypeScript to #1*. GitHub Blog, 28 out. 2025. Métrica: número de desenvolvedores distintos por mês que enviaram código na linguagem; período de 01/09/2024 a 31/08/2025. Disponível em: <https://github.blog/news-insights/octoverse/octoverse-a-new-developer-joins-github-every-second-as-ai-leads-typescript-to-1/>. Acesso em: 25 ago. 2026.

GITHUB. *GitHub GraphQL API documentation*. Fonte de todos os dados minerados (idade, pull requests, releases, atualização, linguagem e issues dos repositórios). Disponível em: <https://docs.github.com/en/graphql>. Acesso em: 25 ago. 2026.